REINFORCEMENT LEARNING

Lecture 9 – Policy Gradient (2)

Gyeongsik Moon
Visual Computing and AI Lab
Korea University

Lecture credit: Emma Brunskill

1. Fix PG with Temporal Structure

2. Fix PG with Baseline

3. Fix PG with Bootstrapping

1. Fix PG with Temporal Structure

Likelihood Ratio / Score Function Policy Gradient

$$\nabla_\theta V(\theta) \approx (1/m) \sum_{i=1}^{m} R(\tau^{(i)}) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)})$$

- Unbiased but very noisy
- Fixes that can make it practical
    - **Temporal structure**
    - Baseline
    - Use Bootstrapping instead of Monte Carlo

Why we introduce ‘Temporal Structure’?

We can eliminate the "noise" from future actions that have no causal link to that reward.

Policy Gradient: Use Temporal Structure

- Previously:

$$\nabla_\theta \mathbb{E}_\tau[R] = \mathbb{E}_\tau\left[\left(\sum_{t=0}^{T-1} r_t\right)\left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)\right)\right]$$

- We can repeat the same argument to derive the gradient estimator for a single reward term $r_{t'}$.

$$\nabla_\theta \mathbb{E}[r_{t'}] = \mathbb{E}\left[r_{t'} \sum_{t=0}^{t'} \nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$

- Causality Constraint: Current actions cannot affect past rewards; therefore, $\nabla_\theta \log \pi_\theta(a_t|s_t)$ should not be scaled by $r_{t'}$ where $t' < t$.
- Irrelevant Noise Removal: By focusing on the gradient for a single reward term $r_{t'}$, we can eliminate the "noise" from future actions that have no causal link to that reward.
- For example, ignore $r_0 \log \pi(a_{100}|s_{100})$

Policy Gradient: Use Temporal Structure

- Previously:

$$\nabla_\theta \mathbb{E}_\tau[R] = \mathbb{E}_\tau \left[\left(\sum_{t=0}^{T-1} r_t \right) \left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \right) \right]$$

- We can repeat the same argument to derive the gradient estimator for a single reward term $r_{t'}$.

$$\nabla_\theta \mathbb{E}[r_{t'}] = \mathbb{E} \left[r_{t'} \sum_{t=0}^{t'} \nabla_\theta \log \pi_\theta(a_t|s_t) \right]$$

- Summing this formula over t, we obtain

$$\begin{aligned} \nabla_\theta V(\theta) = \nabla_\theta \mathbb{E}[R] &= \mathbb{E} \left[\sum_{t'=0}^{T-1} r_{t'} \sum_{t=0}^{t'} \nabla_\theta \log \pi_\theta(a_t|s_t) \right] \\ &= \mathbb{E} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t, s_t) \sum_{t'=t}^{T-1} r_{t'} \right] \end{aligned}$$

Policy Gradient: Use Temporal Structure

$$\nabla_\theta V(\theta) = \nabla_\theta \mathbb{E}[R] = \mathbb{E}\left[\sum_{t'=0}^{T-1} r_{t'} \sum_{t=0}^{t'} \nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$
$$= \mathbb{E}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t, s_t) \sum_{t'=t}^{T-1} r_{t'}\right]$$

Recall for a particular trajectory $\tau^{(i)}$, $\sum_{t'=t}^{T-1} r_{t'}^{(i)}$ is the return $G_t^{(i)}$

$$\nabla_\theta \mathbb{E}[R] \approx (1/m) \sum_{i=1}^{m} \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t, s_t) G_t^{(i)}$$

- Reward-to-go (G_t): The sum of rewards obtained starting from time t (not from time 0) until the end of the trajectory.
- Policy Gradient with Causality: Policy parameters are updated proportional to the score function $\nabla_\theta \log \pi_\theta$
- Variance Reduction: Significantly reduces noise in gradient estimation by discarding irrelevant past rewards that the current action cannot influence.

Monte-Carlo Policy Gradient (REINFORCE)

Leverages likelihood ratio / score function and temporal structure

$$\Delta\theta_t = \alpha \nabla_\theta \log \pi_\theta(s_t, a_t) G_t$$

<u>REINFORCE:</u>
Initialize policy parameters θ arbitrarily
for each episode $\{s_1, a_1, r_2, \cdots, s_{T-1}, a_{T-1}, r_T\} \sim \pi_\theta$
  do for $t = 1$ to $T - 1$ **do**
    $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(s_t, a_t) G_t$
  endfor
endfor
return θ

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning, 8(3), 229-256.

2. Fix PG with Baseline

Likelihood Ratio / Score Function Policy Gradient

$$\nabla_\theta V(\theta) \approx (1/m) \sum_{i=1}^{m} R(\tau^{(i)}) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)})$$

- Unbiased but very noisy
- Fixes that can make it practical
  - Temporal structure
  - **Baseline**
  - Use Bootstrapping instead of Monte Carlo

Why we introduce Baseline?

Objectives of Introducing a Baseline $b(s)$

- Variance Reduction: The primary goal is to lower the high variance of the policy gradient estimator, leading to more stable and faster convergence.
- Relative Performance Assessment: Shifts the focus from absolute reward values to relative advantages, reinforcing actions only if they perform better than the average expectation.
- Balanced Gradients: Prevents the "all-positive reward" problem where every action probability increases regardless of its relative quality, by centering the learning signal.

Policy Gradient: Introduce Baseline

- Reduce variance by introducing a *baseline* $b(s)$

$$\nabla_\theta \mathbb{E}_\tau[R] = \mathbb{E}_\tau \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi(a_t|s_t;\theta) \left(\sum_{t'=t}^{T-1} r_{t'} - b(s_t) \right) \right]$$

- For any choice of b, gradient estimator is unbiased.
- Near optimal choice is the expected return,

$$b(s_t) \approx \mathbb{E}[r_t + r_{t+1} + \cdots + r_{T-1}]$$

- Interpretation: increase logprob of action a_t proportionally to how much returns $\sum_{t'=t}^{T-1} r_{t'}$ are better than expected

Baseline $b(s)$ Does Not Introduce Bias–Derivation

$$
\begin{aligned}
&\mathbb{E}_\tau[\nabla_\theta \log \pi(a_t|s_t;\theta) b(s_t)] \\
&= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} \left[\mathbb{E}_{s_{(t+1):T}, a_{t:(T-1)}}[\nabla_\theta \log \pi(a_t|s_t;\theta) b(s_t)]\right] \text{ (break up expectation)} \\
&= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} \left[b(s_t)\mathbb{E}_{s_{(t+1):T}, a_{t:(T-1)}}[\nabla_\theta \log \pi(a_t|s_t;\theta)]\right] \text{ (pull baseline term out)} \\
&= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} \left[b(s_t)\mathbb{E}_{a_t}[\nabla_\theta \log \pi(a_t|s_t;\theta)]\right] \text{ (remove irrelevant variables)} \\
&= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} \left[b(s_t) \sum_a \pi_\theta(a_t|s_t) \frac{\nabla_\theta \pi(a_t|s_t;\theta)}{\pi_\theta(a_t|s_t)}\right] \text{ (likelihood ratio)} \\
&= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} \left[b(s_t) \sum_a \nabla_\theta \pi(a_t|s_t;\theta)\right] \\
&= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} \left[b(s_t) \nabla_\theta \sum_a \pi(a_t|s_t;\theta)\right] \\
&= \mathbb{E}_{s_{0:t}, a_{0:(t-1)}} [b(s_t)\nabla_\theta 1] \text{ (probability: sum to 1)} \\
&= \mathbb{E}_{s_0:t, a_0:(t-1)} [b(s_t) \cdot 0] = 0 \text{ (derivative of constant is zero)}
\end{aligned}
$$

Baseline $b(s)$ Does Not Introduce Bias–Derivation

- $\text{Policy Gradient Theorem}$
- $\nabla J \approx \mathbb{E}[\nabla \log \pi \cdot G]$
- $\nabla J \approx \mathbb{E}[\nabla \log \pi \cdot (G - b)] = \underbrace{\mathbb{E}[\nabla \log \pi \cdot G]}_{\text{Original Direction}} - \underbrace{\mathbb{E}[\nabla \log \pi \cdot b]}_{\text{Zero (0)}}$

- Same Direction: Since $\mathbb{E}[\nabla \log \pi \cdot b] = 0$, the overall gradient direction (Expectation) remains identical to the original objective.
- Lower Magnitude: Subtracting b centers the learning signal around zero, reducing the absolute magnitude of the updates.
- Stability: Smaller, centered updates mean the policy doesn't "over-correct" or swing wildly, leading to much faster and more reliable convergence.

"Vanilla" Policy Gradient Algorithm

Initialize policy parameter θ, baseline b
for iteration$=1, 2, \cdots$ **do**
  Collect a set of trajectories by executing the current policy
  At each timestep t in each trajectory τ^i, compute
    Return $G_t^i = \sum_{t'=t}^{T-1} r_{t'}^i$, and
    Advantage estimate $\hat{A}_t^i = G_t^i - b(s_t)$.
  Re-fit the baseline, by minimizing $\sum_i \sum_t ||b(s_t) - G_t^i||^2$,
  Update the policy, using a policy gradient estimate $\hat{g}$,
    Which is a sum of terms $\nabla_\theta \log \pi(a_t|s_t, \theta)\hat{A}_t$.
    (Plug $\hat{g}$ into SGD or ADAM)
endfor

From Lecture 8

$$\nabla_\theta V(\theta) \approx \hat{g} = (1/m)\sum_{i=1}^{m} R(\tau^{(i)})\nabla_\theta \log P(\tau^{(i)};\theta)$$

$$= (1/m)\sum_{i=1}^{m} R(\tau^{(i)})\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)})$$

"Vanilla" Policy Gradient Algorithm

- Two-Network Architecture: Modern policy gradient methods typically maintain two separate functional approximators (often two distinct neural networks).
- Policy Network (π_θ): Learns the best action strategy to maximize returns.
- Value Network (V_ϕ or b): Learns to estimate the expected return of the current state, serving as a stable baseline.
- The Birth of Actor-Critic: This structure is the fundamental precursor to Actor-Critic methods, where the "Critic" (Baseline) evaluates the "Actor" (Policy).

Choosing the Baseline: Value Functions

- Recall Q-function / state-action-value function:

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[r_0 + \gamma r_1 + \gamma^2 r_2 \cdots | s_0 = s, a_0 = a\right]$$

- State-value function can serve as a great baseline

$$\begin{aligned} V^{\pi}(s) &= \mathbb{E}_{\pi}\left[r_0 + \gamma r_1 + \gamma^2 r_2 \cdots | s_0 = s\right] \\ &= \mathbb{E}_{a\sim\pi}[Q^{\pi}(s,a)] \end{aligned}$$

3. Fix PG with Bootstrapping

Likelihood Ratio / Score Function Policy Gradient

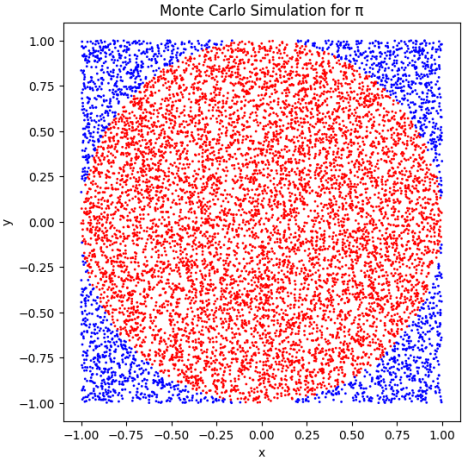
- Policy gradient:

$$\nabla_\theta \mathbb{E}[R] \approx (1/m) \sum_{i=1}^{m} \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t, s_t)(G_t^{(i)} - b(s_t))$$

- Fixes that improve simplest estimator
  - Temporal structure (shown in above equation)
  - Baseline (shown in above equation)
  - **Use Bootstrapping instead of Monte Carlo**

Monte Carlo (MC) Sampling

- Definition: A broad class of computational algorithms that rely on repeated random sampling to obtain numerical results.
- Core Principle: It uses randomness to solve problems that might be deterministic in principle or too complex to solve analytically.
- Law of Large Numbers: As the number of random samples increases, the average of the results converges to the true expected value.



MC vs. TD

- Monte Carlo (MC)
  - A method of estimating values by averaging the actual returns (G_t) from multiple completed trajectories.
  - Learning occurs only after reaching the terminal state (T).
  - Relies strictly on real experience, but suffers from High Variance due to cumulative noise.
- Temporal Difference (TD)
  - A method that updates value estimates based on other learned estimates, without waiting for the final outcome (Bootstrapping).
  - Learning occurs online at every single time step (t) using $r_t + \gamma V(s_{t+1})$
  - Much more stable and efficient, though it introduces Bias during early learning phases.

Choosing the Target

- The Dilemma: We must choose between a noisy but truthful target (G_t) and a stable but potentially "lying" target (Bootstrapping).
- Monte Carlo (MC): High Variance, Zero Bias. Safe but painfully slow to converge.
    - Summation of true rewards from states timesteps.
    - MC Target: $G_t = r_t + r_{t+1} + r_{t+2} + \cdots$
- Bootstrapping (TD): Low Variance, High Bias. Fast and stable, but risks converging to a wrong solution if the approximation is poor.
    - Summation of true reward of a single timestep and function approximation
    - TD Target: $r_t + \gamma V(s_{t+1})$
- Function Approximation: Using a neural network to estimate values naturally introduces bias but is essential for handling large state spaces.

Actor-critic Methods

- Estimate of V/Q is done by a **critic**
- **Actor-critic** methods maintain an explicit representation of policy and the value function, and update both
- A3C (Mnih et al. ICML 2016) is a very popular actor-critic method

Policy Gradient Formulas with Value Functions

- Recall:

$$\nabla_\theta \mathbb{E}_\tau[R] = \mathbb{E}_\tau\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi(a_t|s_t;\theta)\left(\sum_{t'=t}^{T-1} r_{t'} - b(s_t)\right)\right]$$

$$\nabla_\theta \mathbb{E}_\tau[R] \approx \mathbb{E}_\tau\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi(a_t|s_t;\theta)\left(Q(s_t, a_t; \boldsymbol{w}) - b(s_t)\right)\right]$$

- Letting the baseline be an estimate of the value V, we can represent the gradient in terms of the state-action advantage function

$$\nabla_\theta \mathbb{E}_\tau[R] \approx \mathbb{E}_\tau\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi(a_t|s_t;\theta)\hat{A}^\pi(s_t, a_t)\right]$$

where the advantage function $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$

Policy Gradient Formulas with Value Functions

$$\nabla_\theta \mathbb{E}_\tau[R] = \mathbb{E}_\tau \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi(a_t|s_t;\theta) \left(\sum_{t'=t}^{T-1} r_{t'} - b(s_t) \right) \right]$$

$$\nabla_\theta \mathbb{E}_\tau[R] \approx \mathbb{E}_\tau \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi(a_t|s_t;\theta) \hat{A}^\pi(s_t, a_t) \right]$$

- From Returns to Estimates: Transitioning from empirical returns (G_t) to learned value functions (Q and V).
- Actor-Critic Foundation: By replacing the raw return with a learned Q-function, we move into the Actor-Critic framework.
- Generalized Advantage: Expressing the gradient in terms of $\widehat{A^\pi}(s,a)$ allows for more flexible and stable updates using Bootstrapping.

Choosing the Target: N-step estimators

$$\nabla_\theta V(\theta) \approx (1/m)\sum_{i=1}^{m}\sum_{t=0}^{T-1} R_t^i \nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)})$$

Note that critic can select any blend between TD and MC estimators for the target to substitute for the true state-action value function.

$$\hat{R}_t^{(1)} = r_t + \gamma V(s_{t+1})$$

$$\hat{R}_t^{(2)} = r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) \qquad \cdots$$

$$\hat{R}_t^{(\text{inf})} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$$

If subtract baselines from the above, get advantage estimators

$$\hat{A}_t^{(1)} = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$$\hat{A}_t^{(\text{inf})} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+1} + \cdots - V(s_t)$$

Check Your Understanding: Blended Advantage Estimators

$$\nabla_\theta V(\theta) \approx (1/m) \sum_{i=1}^{m} \sum_{t=0}^{T-1} R_t^i \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)})$$

If subtract baselines from the above, get advantage estimators

$$\hat{A_t^{(1)}} = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$$\hat{A}_t^{(\text{inf})} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+1} + \cdots - V(s_t)$$

Select all that are true

$\hat{A}_t^{(1)}$ has low variance & low bias.

$\hat{A}_t^{(1)}$ has high variance & low bias.

$\hat{A}_t^{(\infty)}$ low variance and high bias.

$\hat{A}_t^{(\infty)}$ high variance and low bias.

"Vanilla" Policy Gradient Algorithm

Initialize policy parameter θ, baseline b
for iteration$=1, 2, \cdots$ **do**
  Collect a set of trajectories by executing the current policy
  At each timestep t in each trajectory τ^i, compute
    Advantage estimate $\hat{A}^n_{it}$
  Update the policy, using a policy gradient estimate $\hat{g}$,
    Which is a sum of terms $\nabla_\theta \log \pi(a_t|s_t, \theta)\hat{A}^n_{it}$.
    (**Plug $\hat{g}$ into SGD or ADAM**)
endfor

Note, we can choose which blended estimator $\hat{A}^n$ to use

Current Summary of Benefits of Policy-Based RL

Advantages:

- Better convergence properties
- Effective in high-dimensional or continuous action spaces
- Can learn stochastic policies

Disadvantages:

- Typically converge to a local rather than global optimum
- Evaluating a policy can be inefficient and high variance (though baseline and temporal structure helps)